A change on one side.
A consequence on the other.

Harbour.Space Bangkok | Software Engineering | 5 October 2026

What we did in class

We distributed Product Owner, Architect and SRE responsibilities within teams and continued preparing the codebase for extraction. We revisited contracts and breaking changes through a booking-creation example: changing the arguments described in the API specification can break the frontend that calls it. We also accepted and merged two ADRs and began work arising from their consequences: moving reporting to Grafana and separating the frontend. Work started does not mean every consequence has been implemented or verified.

1. Roles make responsibility visible

Product Owner: helps clarify the user need, priorities and intended outcome.
Architect: helps the team reason about boundaries, interfaces and design trade-offs.
SRE (Site Reliability Engineering responsibility): helps the team make running, checking and recovering the system explicit.
These are accountabilities within a team. Everyone still contributes to implementation, review and verification; a role does not make one person the only person allowed to do that work.

2. A contract is a promise a caller relies on

An interface contract states what a caller supplies and what it can expect: inputs, meanings, results and failure behaviour. For booking creation, the frontend is a **consumer** and the backend is a **provider**. They must agree on more than a function or endpoint name. Required fields, types, time formats and error responses can all affect whether they work together.

An **OpenAPI description** records an HTTP API in a machine-readable form, including operations, request bodies and responses. It helps both teams work from the same description; it does not prove that the running implementation follows it. See OpenAPI Specification: introduction and request/response objects.

3. A breaking change invalidates an existing expectation

Ask: **will a previously valid caller still work, with the meaning it relied on?** A change can break a consumer even when the backend starts successfully and its own tests pass. In our class example, the booking arguments connect the API definition directly to the frontend request. Updating the definition alone does not update that caller.

Reference: three documents, three questions

Business rules: what must happen.
Tech spec: how the teams will implement and check it.
ADR (Architecture Decision Record): why this design was chosen over alternatives, and what costs they accept.

Use this distinction to connect the accepted ADRs on page 3 to the rules and implementation work. Link the documents rather than repeating the same information.

Trace the caller before changing the promise.

4. Work through the booking example

Illustrative reconstruction, not the exact field names from class: suppose the existing frontend sends `start_time`. A new backend requires `starts_at` and no longer accepts the old name. The frontend still sends its old request, so the required new field is missing. The booking can now fail even though the member filled in the same form correctly.

Trace: unchanged form > old request field > new required field missing > booking rejected.
The contract disagreement sits between the caller and provider. Changing an internal variable while preserving the accepted request would be a different case.

5. Coordinate a change and check the transition

Before changing the interface, identify its consumers and compare an existing request with the proposed one. Agree how callers will move to the new contract. One possible transition is to accept the old input temporarily while consumers update; that needs explicit rules if both forms are supplied. Another is a coordinated update with a known compatibility window and recovery plan. These are options to evaluate, not a policy agreed in this class.

Check the **old caller against the new provider**, the updated caller against its intended provider, and a useful invalid-input case. Then exercise the booking flow through the frontend. A schema check, backend test and browser journey answer different questions. Record what actually ran and what remains unverified. For compatibility principles, see Google AIP-180: Backwards compatibility.

6. Preparation for extraction is already engineering work

Clarifying ownership and reducing hidden dependencies makes a future split easier to reason about. Moving code into another module does not by itself establish an independent service. Follow one operation across the boundary: who owns the rule, which data does it need, what does the caller rely on, and what happens if the operation cannot complete? Explain the benefit and remaining coupling before claiming the split is ready.

Self-checks

1. Why can renaming a required request field break an unchanged frontend?
2. Does updating OpenAPI prove that both implementations agree?
3. Which check would catch an old caller failing against a new provider?
4. How do the three team responsibilities help with this change?

Answer cues: the old request no longer meets the promise; documentation is not runtime evidence; send the old request to the new provider and check the result, then check the user journey; clarify intended behaviour, design the transition, and plan verification/recovery together. These are reflection prompts, not an extra assignment.

Accepted decisions create follow-up work.

CLASS EXAMPLES / TWO MERGED ADRs

7. Move reporting to Grafana

Reporting helps people understand the business, but Spacey does not need to build every reporting screen itself. ADR 0001 chooses Grafana for read-only reporting, using an available system so development can focus on the member journey. Alternatives were retaining the application dashboard or building a separate custom reporting service.

Expected benefit: less custom reporting UI to maintain.
Costs and obligations: Grafana becomes a dependency; reporting queries and application data must remain compatible. Read-only access must be scoped, and the replacement reports must agree with the existing metric meanings. Moving a chart does not remove responsibility for its numbers.

Check to perform: compare both reports using the same data and time window. A failed query must not appear as a valid zero. Preserve the existing metrics API until its callers are separately assessed.

8. Separate the browser client

ADR 0002 chooses a separate frontend repository so compatible UI changes can be reviewed and released independently. Keeping the UI in the monolith or using a monorepo with separate deployments were alternatives. The decision also enables removal of obsolete server-rendered pages and duplicated form paths while retaining the JSON API and backend business rules.

Expected benefit: clearer delivery ownership and fewer presentation paths in the backend.
Costs and obligations: separate builds and deployments, explicit API compatibility, and coordinated changes when a promise changes. Recovery needs a compatible frontend/backend pair; reverting only one side may not restore the journey.

Connection to our contract example: changing booking arguments affects the separately developed frontend. The ADR explains why this boundary was chosen; the contract and checks establish whether both sides still work together.

9. Accepted is a decision status

Both ADRs were accepted and merged, and follow-up work started. Completion and verification remain separate claims. Follow the chain: decision, consequence, work item, observed result. Compare actual results with expected benefits.

Self-check

Choose one ADR. Name an alternative, one expected benefit, one cost, and a check that would tell you whether the change worked. For the frontend, why are fewer lines of code insufficient evidence that the new boundary is better?

Reflection only: connect fewer duplicated paths to a preserved member journey.

Public class sources, pinned at Spacey b4fec381:
ADR 0001: Move reporting to Grafana
ADR 0002: Separate the browser client. The records distinguish accepted decisions from pending implementation and verification.